

EGAD: Extending GPU-Accelerated Databases Beyond Device Memory

Christian Tabbah
University of Toronto
christian.tabbah@mail.utoronto.ca

Angela Demke Brown
University of Toronto
demke@cs.toronto.edu

Ashvin Goel
University of Toronto
ashvin@eecg.toronto.edu

Abstract

GPU databases exploit the GPU’s parallelism and memory bandwidth to achieve high throughput on in-memory workloads. While scaling GPU databases beyond device memory is now well-studied for analytical workloads, the transactional case is not. The state of the art in GPU-accelerated OLTP databases achieves high throughput through deterministic batched execution but is bounded by GPU memory capacity, falling back to CPU execution and losing substantial throughput once the database exceeds VRAM. However, a batch’s working set is typically much smaller than the full database.

We present EGAD, a GPU-accelerated transactional database that keeps its Primary Store in CPU memory and stages each batch’s working set onto the GPU for execution. Like prior GPU-accelerated OLTP systems, EGAD plans concurrency control before executing each batch deterministically and uses dual-versioning. EGAD adds a staging layer that admits the records each batch needs and writes back the records each batch modifies. Each batch’s writes only modify one of the two versions, which EGAD exploits to halve per-record PCIe traffic, run writeback concurrently with the next batch’s execution, and keep the CPU’s Primary Store at most two batches behind the GPU. On a GPU crash, recovery replays at most two batches of logged transaction inputs since the rest resides in CPU memory, where standard durability techniques apply. Our evaluation on TPC-C and YCSB shows that EGAD outperforms CPU-only execution by 1.41x on TPC-C, continues running past the database sizes where GPU-only execution stops, and provides GPU-crash recovery whose throughput cost leaves EGAD still ahead of CPU-only execution. CPU-only execution stays competitive at low skew, while under contention EGAD’s advantage grows to 3.1x on YCSB.

1 Introduction

High-throughput online transaction processing (OLTP) is a foundational requirement of modern data infrastructure. In-memory databases dominate this space [2, 10, 11, 14, 16, 18], and multi-version concurrency control (MVCC) is the standard approach for contended workloads [2, 7, 10, 14, 16], since maintaining multiple versions per record lets readers proceed without blocking writers. However, MVCC traditionally incurs overhead from version search and garbage collection.

Recent GPU-accelerated OLTP systems have addressed this differently. GPUDx [5] and GaccO [1] avoid the over-

head of MVCC by using single-version concurrency control. However, this comes at the expense of the read-write parallelism that MVCC enables when contention is high. Epic [15] takes a multi-versioned approach and eliminates the version-search overhead by pre-computing version locations during a per-batch initialization phase. After this phase, transactions run concurrently on the GPU with no version search and no aborts, synchronizing only where a transaction reads a version written earlier in the same batch.

Despite these advances, two limits constrain what GPU-accelerated OLTP can support in practice, the first of which is capacity. While OLTP databases can be arbitrarily large, GPU memory is bounded (e.g., tens of gigabytes on commodity cards). The second limit is persistence. In-memory OLTP systems can recover from failure through well-established techniques, ranging from write-ahead logging of CPU-memory state to disk [2, 12, 13, 18, 23] to hardware-backed mechanisms such as battery-backed DIMMs that make CPU memory itself non-volatile [6]. However, GPU-only systems have no equivalent convention. This means that existing GPU-accelerated OLTP designs, including Epic, do not provide a recovery mechanism. To overcome the capacity limit, Epic includes a CPU-execution fallback that runs indexing and initialization on the GPU but moves execution to the CPU. Yet this does not address the persistence problem, since neither mode provides a path from GPU memory to durable storage.

The challenge of scaling GPU databases beyond device memory is not unique to OLTP. It has been studied extensively in the analytical setting. Vortex [22] streams data through the GPU on demand, with no persistent on-GPU caching, while Yinan-Li *et al.* [9] use CPU-side filtering to shrink the volume of data transferred to the GPU before joins. Lancelot [21] caches hot data across multiple GPUs with eviction back to the CPU. While these designs have established that out-of-VRAM scaling is feasible for analytical workloads, they do not transfer to OLTP. They assume read-only scans, immutable tables, and pipelined access patterns, none of which hold for OLTP workloads. Transactional workloads exhibit random reads and writes, mutate the database, and need their modifications propagated back to CPU memory. Streaming and pre-filtering provide none of these guarantees.

GPU-accelerated OLTP is fast under most contention levels but capacity-bound, so the challenge becomes scaling beyond VRAM without losing that advantage. We observe that, within any single batch (e.g., 100K transactions), the constraint is far less severe than the overall database size suggests. The

records actually referenced by transactions in a single batch (what we call the per-batch working set) are typically much smaller than the full database for OLTP workloads, especially when there is access skew. If only this working set must fit in GPU memory, a system should be able to run on databases that exceed VRAM while preserving the benefits of GPU acceleration.

This observation motivates our hybrid approach. We present EGAD, a GPU-accelerated transactional database that keeps the database in CPU memory as the Primary Store and stages just the working set onto the GPU at the start of each batch. The records modified during a batch are flushed back to CPU while the next batch executes. The Primary Store therefore stays at most two batches behind the GPU. The executing batch has not yet flushed, and the previous batch’s flush may still be completing. Furthermore, the Primary Store can be extended to durable storage through any of the techniques above.

EGAD inherits Epic’s deterministic batched execution model and extends it in three ways. A staging layer mediates between the CPU Primary Store and a GPU-resident cache we call the Execution Cache, a writeback path propagates modified records back to the Primary Store, and an indexing modification translates between separate CPU and GPU identifier spaces at each batch boundary. The Execution Cache is partitioned per table, so each table has its own staging pipeline and capacity budget. This lets each table’s pipeline run independently of the others and allows the cache capacity to be allocated where the workload needs it.

Realizing this insight requires taking advantage of deterministic batched execution. Since each batch’s read-write set is known before execution begins, EGAD can prefetch the records each batch needs into the Execution Cache, after which execution runs at full GPU speed against fully-resident data. From this, two challenges follow. First, the prefetch set must be computed efficiently at each batch boundary, within the millisecond budget that preserves native GPU execution throughput. Second, when the prefetch set exceeds the Execution Cache, the eviction set must be selected efficiently.

Writeback to the CPU Primary Store presents a third challenge. While records modified during a batch must be propagated back to the Primary Store, doing so synchronously at batch boundaries adds device-to-host transfers and CPU-side scatter to each batch’s tail latency. This erodes the throughput advantage, forcing the writeback to be done asynchronously instead. This requires the writeback to be coordinated with the next batch’s prefetch over the shared Primary Store and the GPU’s transfer path.

This paper makes three main contributions. First, we contribute a hybrid staging architecture where the CPU memory holds the primary database and the GPU memory holds a working-set-sized cache that is refreshed at each batch boundary, enabling GPU-accelerated OLTP databases to scale beyond GPU memory at native GPU execution throughput. Sec-

ond, we exploit Epic’s dual-version storage in two ways. The admission ships only the version that will be read, and the writeback only the version that was modified, halving per-record PCIe traffic in each direction. Because each batch’s writes touch only one of the two version slots, the writeback runs concurrently with the next batch’s execution. Third, we contribute a recovery mechanism for GPU crashes. EGAD keeps the CPU Primary Store at most two batches behind the GPU, so recovery replays at most two batches of logged transaction input, with the rest residing durably in CPU memory through standard durability techniques.

The rest of this paper is organized as follows. §2 reviews Epic’s deterministic batched MVCC and dual-version storage along with related OLTP and GPU-database work. §3 presents EGAD’s design. §4 describes how the stager and its supporting subsystems are implemented. §5 evaluates EGAD on YCSB and TPC-C. §6 concludes.

2 Background and Related Work

This section reviews the literature EGAD builds on (deterministic batched OLTP, multi-version concurrency control, GPU architecture for OLTP, and host-device data movement) before summarizing Epic, the system EGAD extends.

2.1 Deterministic Batched OLTP

Deterministic transactional database systems group transactions into batches and assign each batch a serial execution order before execution begins. We refer to such a batch as an *epoch* and use this term for the rest of the paper.

Determinism in a database means that the read and write sets of each transaction are known in advance. One-shot stored procedures satisfy this requirement directly, since their parameters determine which records they touch. For more general transactions, the system can run reconnaissance queries [17] to identify their read-write sets prior to execution.

A line of deterministic in-memory databases has explored this design space with differing conflict-resolution methods. Calvin [17] uses a centralized lock manager. PWV [4] performs per-core dependency analysis. Bohm [3] pre-allocates versions in a partitioned MVCC initialization phase. Caracal [16] uses shared-memory MVCC initialization to better handle skewed workloads. Epic [15] performs concurrency-control planning and transaction execution on the GPU. All five resolve conflicts at planning time, allowing transactions to run without runtime concurrency-control aborts. Aria [11] takes a different approach, avoiding the requirement to know read-write sets in advance and instead detecting conflicts deterministically at commit.

Determinism and epoch batching are a natural fit for GPU-accelerated OLTP. The work within an epoch parallelizes naturally across many threads, and resolving conflicts at planning time means execution proceeds without retries.

2.2 Multi-Version Concurrency Control

Multi-version concurrency control (MVCC) is a technique that maintains multiple versions of each record so that readers can access committed values without blocking writers. It allows writers to create new versions in parallel while readers access older records, providing high read-write parallelism under contention.

Unfortunately, this degrades performance in three ways. First, reads must search the version chain to locate the appropriate version, adding traversal overhead. Second, each commit allocates a new version, consuming memory and requiring garbage collection. Third, garbage collection itself is non-trivial when versions form linked structures with arbitrary access timing. Recent in-memory MVCC systems [2, 7, 10, 14] have substantially reduced these costs, and Wu *et al.* [19] provide an empirical comparison showing that single-version systems often outperform MVCC under low contention but lose under high contention. Rather than simply reduce these costs, Epic [15] eliminates version search entirely by pre-computing version locations during a per-epoch planning phase and minimizes garbage-collection cost by reclaiming temporary versions in $O(1)$ time at epoch boundaries (§2.5). EGAD reuses Epic’s MVCC implementation.

2.3 GPU Architecture for OLTP

Modern NVIDIA GPUs execute work on streaming multiprocessors (SMs), each running threads in groups of 32 called warps. The threads in a warp execute in lockstep under single-instruction, multiple-thread (SIMT) semantics. They share a program counter and execute the same instruction together, but each thread can operate on different data. When threads in a warp access contiguous memory addresses, the accesses are coalesced into a single memory transaction, while non-contiguous accesses are serialized, wasting memory bandwidth.

A warp can suffer from branch divergence when its threads need to execute different code paths. The lockstep model forces the GPU to run each divergent path in turn, so the warp pays the cost of every path it takes. OLTP workloads are highly susceptible because each transaction has data-dependent branches, and the natural approach of running one transaction per thread puts many different transactions in the same warp. Epic mitigates branch divergence with *warp-cooperative execution*, in which a single transaction is assigned to an entire warp so that each thread follows the same control path. The threads then cooperate on memory operations, so record copies coalesce into single memory requests rather than wasting the warp’s bandwidth. EGAD inherits this execution model from Epic and applies the same warp-cooperative pattern across other memory-intensive kernels (§3).

2.4 Host-Device Data Movement

Modern GPUs hold tens of gigabytes of device memory (VRAM), while host DRAM is typically several times to one or two orders of magnitude larger. The two are connected via a PCIe bus that limits host-device data movement. Systems whose data exceeds VRAM capacity must manage this movement explicitly.

Applications dispatch transfers via asynchronous memcpy, which enqueues work onto the GPU’s *copy engine*. NVIDIA GPUs expose one or two physical copy engines per direction. With a single engine per direction, every transfer in that direction queues through it in FIFO order regardless of CUDA stream or context. The host-to-device (H2D) and device-to-host (D2H) engines are independent, so transfers in opposite directions can overlap. The copy engine in each direction is therefore a single point of contention for systems overlapping multiple transfers in that direction.

Host-side memory modes determine whether a DMA transfer runs asynchronously. With *pinned* memory, pages are locked against swap-out, so the copy engine transfers directly. *Pageable* memory blocks the host thread while the driver copies into an internal staging buffer. *Mapped* memory makes a region of host DRAM directly accessible to GPU kernels via PCIe load and store, bypassing the copy engine but paying per-access latency, so it is practical only for small transfers that would otherwise queue behind a bulk transfer. EGAD uses pinned memory for bulk record transfers and mapped memory for per-epoch scalar readbacks (§3).

Concurrency in CUDA is organized around *streams*. Operations within a stream run in FIFO order, while operations on different streams may run concurrently, the primary mechanism for overlapping kernels with transfers and with other kernels. However, the driver can serialize CUDA calls issued concurrently from multiple host threads, so multi-threaded applications cannot always achieve the API-call concurrency they expect. EGAD pipelines transfers and flushes across multiple streams to hide latency.

NUMA affects host-side bandwidth on multi-socket systems. Accesses crossing socket boundaries traverse a slower interconnect, so where a system places its worker threads relative to the memory they touch shapes both bandwidth and core contention. EGAD places its staging and writeback threads deliberately (§3).

2.5 Epic

EGAD builds on Epic [15], a recent GPU-accelerated, deterministic, multi-version OLTP database. We inherit its concurrency-control planning and warp-cooperative execution kernel. To support a CPU Primary Store with a GPU staging cache, we modify and extend most of Epic’s other components, and add new staging and writeback layers (§3).

Epic’s storage holds two versions per record in a dense

table, with a separate scratchpad for the temporary versions produced during an epoch, which only transactions later in the same epoch read (*tempVer* locations). We refer to the two table slots as *prevVer* and *currVer*. Together, they hold the records’ two most recent durable versions. Each slot carries the epoch ID that last wrote it, and the two are distinguished by comparing these tags. The newer tag marks *prevVer*, the record’s latest committed version, while the slot with the older tag serves as *currVer*, the slot that the current epoch’s final write claims. A writer publishes a version by writing its data, issuing a memory fence, and then setting the slot’s epoch tag, so a version becomes visible only once complete. An epoch’s writes to a record therefore land in exactly one of its two table slots, a property EGAD’s writeback and recovery build on (§3). Within an epoch, writes accumulate in the scratchpad, and only the final write to a record is promoted to the table. At the end of each epoch, the scratchpad is reset in $O(1)$ time, eliminating the per-version garbage-collection cost that traditional MVCC schemes pay.

Each epoch in Epic begins with an *index transfer* that ships the epoch’s transaction inputs from CPU memory to GPU memory, then passes through three GPU-resident phases. The first, *Indexing*, converts user-supplied keys into record IDs via a hash table that persists across epochs. Each epoch performs bulk parallel lookups against it, and the table itself mutates only when keys are inserted or deleted. The second, *MVCC Initialization*, sorts the read-write operations in the epoch by record ID and transaction ID and pre-computes, for each operation, the version location it will read or write, using parallel prefix and postfix passes over the sorted operations. Each location names the record’s *prevVer*, its *currVer*, or a *tempVer* in the scratchpad, so every intra-epoch read is matched to the write that produces its value. By the time the *Execution* phase begins, every transaction already knows exactly where to read and write. This eliminates version search and any abort-based coordination during execution, but not all synchronization. A read that depends on a write from the same epoch spins on that version’s epoch ID until the writer publishes it, while reads of versions written in earlier epochs proceed without synchronization [15]. Each transaction is run by a single warp using warp-cooperative execution (§2.3).

Epic optionally splits records by field. A record with k fields becomes k logical records, each with its own dual-version storage. Single-field updates then modify only the relevant logical record, avoiding the cost of copying $k - 1$ untouched fields. Field splitting is a configurable option, which the evaluation exercises for YCSB (§5.4.3); neither system uses it for TPC-C.

Epic supports two execution modes, *GPU-only* and *CPU-only*. In GPU-only mode, all three phases run on the GPU, requiring the database to fit in VRAM. In CPU-only mode, indexing and initialization run on the GPU, but execution moves to the CPU and reads versions from CPU memory. CPU-only mode supports databases larger than VRAM but

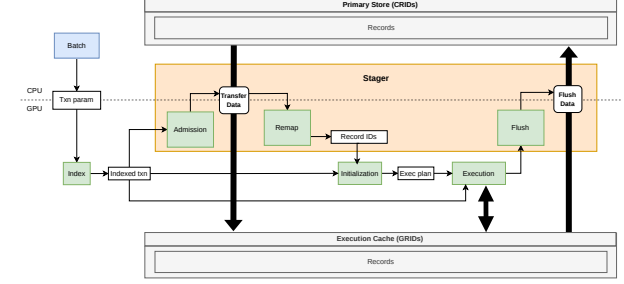


Figure 1: EGAD Architecture

pays a substantial throughput cost (§1).

3 Design and Implementation

EGAD is a GPU-accelerated transactional database that builds on Epic’s deterministic batched execution. Its design separates storage from execution across two stores, with a per-epoch stager moving records between them.

Figure 1 depicts the high-level architecture. The *Primary Store* on the CPU holds the full database, addressed by CPU record IDs (*CRIDs*). The *Execution Cache* on the GPU holds the records recent epochs have touched, addressed by GPU record IDs (*GRIDs*). *CRIDs* identify records permanently, while *GRIDs* identify cache entries. The mapping from *CRID* to *GRID* is what changes as records are admitted and evicted. Each epoch passes through the three pipeline phases inherited from Epic. The *indexing phase* converts user keys to *CRIDs* via a GPU hash table. The *MVCC initialization phase* sorts the operations and pre-computes the version locations each operation will read or write. The *execution phase* runs every transaction in the epoch in parallel against those locations.

EGAD’s main addition is the *stager* (the orange region in Figure 1) with three stages. The *admission stage*, between indexing and MVCC initialization, decides which records the epoch needs in the cache, admits the missing records from the Primary Store, and evicts cached records to make room. Admission and eviction are a single unified operation, with each admitted record overwriting an evicted entry, so the rest of the paper uses the two terms interchangeably. Immediately after, the *remap stage* rewrites each transaction’s *CRIDs* into *GRIDs* so that the MVCC initialization and execution phases can address cached records directly. The *writeback stage*, after execution, propagates the epoch’s modifications back to the Primary Store. EGAD runs the writeback asynchronously by default, off the main thread’s critical path. §§3.3 through 3.7 expand on these stages. §3.8 shows how the Primary Store provides recovery after a GPU failure, §3.9 adds record deletes, and §3.10 collects the design’s limitations.

3.1 Two-Store Design and Identifier Spaces

EGAD addresses records using two separate identifiers. A CRID identifies a record in the Primary Store and persists for the record’s lifetime in the database. A GRID identifies an *entry* in the Execution Cache. Each entry holds the record’s two *version slots* inherited from Epic (§2.5). The cache holds a moving subset of records, so the CRID-to-GRID mapping changes as records are admitted and evicted. During eviction, this separation lets a cache entry be remapped to a different record without affecting how either record is identified in the Primary Store.

The primary index, key-to-CRID, maps user keys to record identities. It is updated only when a key is inserted or deleted, both of which happen in the indexing phase (§3.9). The primary index is implemented as a hash table because user keys are arbitrary types that cannot directly index an array. It lives on the GPU so that the indexing phase can look up an entire epoch’s worth of keys in parallel at kernel speed. A CPU mirror of the primary index, the *Shadow Index*, is maintained for crash recovery (§3.8), and is consulted only after a crash, never during the per-epoch indexing pass.

The cache index, CRID-to-GRID, maps CRIDs to cache entries. It is updated every epoch as records are admitted and evicted, so it is updated much more frequently than the primary index’s insert-driven mutations. The cache index lives on the GPU since the remap stage immediately precedes the execution kernel, which operates entirely on GRIDs. CRIDs are dense bounded integers allocated in sequence by the indexing phase, so the cache index can be implemented as a flat device array indexed directly by CRID. The lookup is a single load with no hashing or chaining, and bulk operations (insert, rename on eviction, mark-dirty) fit naturally as one kernel per operation. The cost is that the array is sized to the total number of records in the database rather than the cache capacity, so VRAM usage scales with database size. This is acceptable at the database sizes EGAD targets, and scaling to an order of magnitude larger would call for a sparse or hierarchical index. The same flat-array pattern applies to other EGAD indices with dense, bounded keyspaces.

A single key-to-GRID index would replace this pair, but at higher cost on the per-epoch inner loop. The primary index is mutated only on inserts, which are workload-driven and typically infrequent. A combined key-to-GRID hash table would instead be mutated on every admission and eviction, which are working-set-driven and far more frequent. EGAD’s two-index design concentrates that per-epoch update traffic in the cache index’s flat array, where each update is a single store rather than a hash-table operation. In practice the two indices see very different update rates. On YCSB-F at the evaluation’s cache ratio, each epoch performs 1 M record operations and inserts no new keys, so the primary index is never updated after load. The cache index updates on every admission and eviction instead. At $\theta = 0.01$, accesses spread

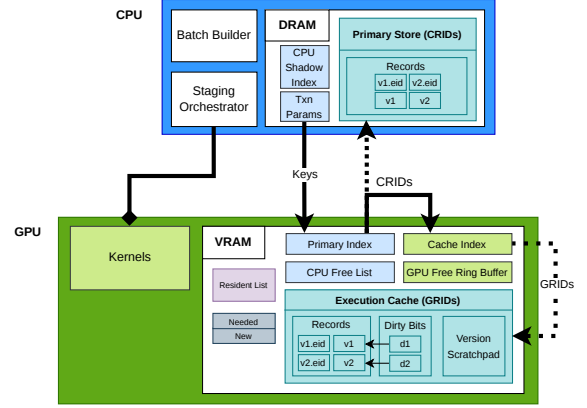


Figure 2: EGAD Storage

nearly uniformly across a table three times the cache’s size, so misses stay common and the cache admits 0.56 M records per epoch even in steady state. Each admission into the full cache also evicts a resident record, giving up to 1.1 M cache-index updates per epoch, more than one per operation (Figure 5). At $\theta = 0.99$ the hot records stay resident and admissions fall to nearly zero. A combined key-to-GRID table would pay every one of those updates as a hash-table operation. The split design pays them as single stores into the flat array.

EGAD also keeps a flat GRID-to-CRID array, the *resident list*, sized to each partition’s cache capacity. Admissions and renames use the cache index, while eviction (§3.4) and the writeback (§3.7) use the resident list. The index pair gives each operation $O(1)$ lookup in its natural direction.

Figure 2 summarizes the layout. The CPU side holds the Primary Store and the primary index’s Shadow Index. The GPU side holds the Execution Cache, the primary index, the cache index, the resident list, and Epic’s per-epoch version scratchpad. Each record in both stores carries two version slots inherited from Epic (§2.5). The two slots, together with the version scratchpad, support MVCC within an epoch and enable asynchronous writeback by guaranteeing that the current epoch’s executor and the previous epoch’s writeback worker touch disjoint slots, as §3.7 shows.

3.2 Per-Table Cache Partitioning

A database’s tables differ in row count, in record size, in access pattern, and in whether they grow through inserts. The simplest design is a unified Execution Cache spanning all tables. The appeal is straightforward sizing from one device-memory budget. The drawback is that a single eviction policy must serve every table, with no way to specialize for tables whose access patterns differ. A unified cache would also need to handle these differing record sizes, either by managing variable-sized slots or by sizing every slot to the largest record and fragmenting tables with smaller records.

Algorithm 1: Working-Set Computation

```
// Input: touched, the epoch's CRIDs emitted by indexing, with
// duplicates. State: cacheIndex[CRID] (holds a GRID or  $\perp$ ),
// evictionProtect[GRID] (cleared at epoch start)
1 Function ComputeWorkingSet (touched):
2   sorted  $\leftarrow$  ParallelSort(touched)
3   needed  $\leftarrow$  UniqueCompact(sorted) // one entry per distinct
   CRID
4   new  $\leftarrow$  []
5   for each crid  $\in$  needed in parallel do
6     g  $\leftarrow$  cacheIndex[crid] // flat-array load
7     if g  $\neq \perp$  then
8       evictionProtect[g]  $\leftarrow$  true // cached, keep this epoch
9     if g  $= \perp$  then
10      append(new, crid) // must be admitted
11 return (needed, new)
```

EGAD partitions the Execution Cache by table instead. Each table gets its own entry range, with two benefits. First, the eviction policy in each partition can be chosen independently, so a workload-aware choice can match each table's access pattern, and tables whose working set fits the partition can stay resident with no eviction at all. Second, the per-epoch pipeline parallelizes naturally across partitions, since each partition owns its own free-list ring buffer, eviction cursor, dirty-flag arrays, and cache index. Admission, eviction, dirty-tracking, and writeback for different tables therefore run independently, and each eviction scan only considers its own partition's entries.

Each partition's capacity is decided at system startup. A deployment grants fully resident tables enough room to hold themselves entirely and bounds the rest to a working set, so fully resident tables incur no per-epoch transfer cost at steady state and only the bounded ones drive the staging pipeline. This static sizing cannot adapt to workload shifts at runtime, but a periodic resize at epoch boundaries, driven by per-partition eviction rates, would fit cleanly within the same staging interface.

3.3 Determining the Working Set

Because execution kernels read records only from the cache, every record the epoch's transactions will touch must be resident in the Execution Cache before execution begins. Deterministic batched execution makes this prefetch set computable exactly, since each transaction's read-write set is known before the epoch starts. The design challenge is to compute this set efficiently on the GPU and to protect records the epoch still needs from being evicted to make room for new ones.

Within each partition, the computation produces two sets that drive the rest of the staging phase. The *needed* set lists every record the epoch will touch. The *new* set lists those records that are not already cached. Producing both together avoids a second pass over the same input.

The computation runs as one bulk pass on the GPU. The

input is each transaction's set of touched CRIDs, produced by the indexing phase. Parallel sort and unique compaction produce the *needed* set. A single lookup against the cache index then separates that set into cached records, whose entries are marked *eviction-protected*, and uncached records, which become the *new* set. The eviction-protected bit is per-entry, cleared at the start of each epoch, and read by the eviction policy when selecting victims (§3.4). Algorithm 1 shows the pass. The sort and the unique compaction are standard data-parallel primitives, and the cached-versus-new split falls out of one flat-array load per distinct CRID, so the computation costs a few bulk kernels regardless of how skewed the epoch's accesses are.

3.4 Unified Admission and Eviction

Admission must bring in every record in the *new* set and, when the cache is full, evict cached records to make room. EGAD does both at once. When a new record is transferred from the Primary Store, it overwrites the entry of a record being evicted, fusing admission and eviction into a single operation. The cache index update invalidates the old CRID's entry and writes the new CRID's entry to the same GRID. No separate writeback step is needed because every overwritten entry is already clean (§3.7.2 explains how).

Algorithm 2 captures admission as a single unified path. The stager pops as many free entries as the ring buffer holds, then picks victim entries for the remaining new records (a count we call the *deficit*) using the partition's eviction policy. For each victim, *bulkRenameCRID* updates the cache index, invalidating the old CRID's entry and writing the new CRID's entry to the same GRID. *install* writes the cache-index mapping for each ring-pop entry. A single bulk transfer then moves every record in *new* from the Primary Store into its assigned entry.

EGAD's eviction policy is a second-chance FIFO scan over the resident list (Algorithm 3). The stager maintains a GRID-space cursor that advances each epoch. The scan starts at the cursor and selects the first *deficit* GRIDs that are neither eviction-protected nor still awaiting writeback (a safety condition under asynchronous writeback, §3.7.2). The scan also skips slots whose *residentList* entry is empty, which prevents FIFO from re-claiming a slot that Algorithm 2 already popped from the free-list ring but has not yet installed. The cursor advances past the farthest entry examined, so a single scan covers a contiguous window whose width is *deficit* plus the number of skipped entries. The per-epoch cost scales with this window rather than with cache capacity. Skipping eviction-protected entries gives the policy a CLOCK-like character. Records that appear in the working set each epoch are protected on each sweep and accumulate in the cache, while records that drop out go unprotected and cycle out on the next sweep.

A history-aware policy like LRU, LFU, or S3-FIFO [20]

Algorithm 2: In-Place Admit and Evict

```
// Inputs: new (CRIDs to admit, computed in Section 3.3) and
// freeSlots (free-list ring count)
1 Function AdmitWithEviction (new, freeSlots):
2   popCount  $\leftarrow \min(|new|, freeSlots)$ 
3   deficit  $\leftarrow |new| - popCount$ 
4   freeGrids  $\leftarrow \text{ringPop}(popCount)$ 
5   (evictGrids, evictCrIds)  $\leftarrow \text{FIFOSelectVictims}(deficit)$ 
   // In-place rename: each evicted CRID's slot now holds a new
   // CRID
6   bulkRenameCRID(evictCrIds, new[0..deficit-1], evictGrids)
7   install(new[deficit..], freeGrids)
   // Single bulk transfer fills both the renamed and the free slots
8   transferCPUtoGPU(new, evictGrids || freeGrids)
```

Algorithm 3: FIFO Victim Selection

```
// State: cursor (advancing GRID position), evictionProtect[GRID]
// (set in Section 3.3), pinned[GRID] (set by in-flight async flush),
// residentList[GRID]
1 Function FIFOSelectVictims (deficit):
2   victims  $\leftarrow []$ 
3   maxOffset  $\leftarrow 0$ 
4   for off  $\leftarrow 0$  to capacity-1 do
5     g  $\leftarrow (cursor + off) \bmod capacity$ 
6     if residentList[g] =  $\perp$  then
7       continue
8     if evictionProtect[g] or pinned[g] then
9       continue
10    append(victims, (g, residentList[g]))
11    maxOffset  $\leftarrow off$ 
12    if |victims| = deficit then
13      break
   // Cursor jumps past the farthest-examined slot, skipping over
   // the protected and pinned slots in one step
14   cursor  $\leftarrow (cursor + maxOffset + 1) \bmod capacity$ 
15   return victims
```

could refine victim selection beyond what the eviction-protect mechanism already does. The eviction-protect bit guarantees that this epoch's working set is preserved, so any improvement must come from exploiting locality across epochs. Since EGAD's epochs contain thousands to hundreds of thousands of transactions (§5.4.4), each epoch's working set is already large, and the additional records a history-aware policy could retain are typically few relative to the per-epoch admission volume, a result we verified by simulating LRU, LFU, S3-FIFO, and Belady-OPT against FIFO. EGAD therefore uses FIFO, which needs only a cursor and two single-bit per-entry flags (eviction-protected and pinned), joined by a third reclaim flag when the workload deletes records (§3.9). LRU and LFU would add per-entry timestamps maintained on every access, and S3-FIFO would add three queues and a frequency counter per entry. For workloads where records are consumed shortly after they are inserted, FIFO's eviction order already matches the order in which the workload stops needing them.

3.5 Tracking Modified Slots

Each epoch's executor modifies exactly one of a record's two version slots (§2.5), so the other version slot's content remains identical to the Primary Store's copy. EGAD therefore marks each modified version slot, rather than each modified record. Tracking modifications at the version-slot level lets the writeback transfer only the changed version slot, halving the per-record D2H volume compared to per-record tracking.

Each stager maintains two GPU arrays of per-version-slot dirty flags, one for each version slot, sized to its partition's capacity. The executor sets a version slot's flag as part of writing that version slot. The writeback phase (§3.7) scans the flags to find modified version slots and clears each one once the version slot's data has reached the Primary Store.

3.6 Remap

The executor addresses cached records by GRID, but the indexing phase emits CRIDs. EGAD bridges them with the remap stage, a translation step at the end of staging that rewrites every operation's CRID into its corresponding GRID. The CRIDs to translate are those in the *needed* set, which admission has just made cache-resident. Running this translation as a separate step, rather than inside the executor, keeps a flat-index load and the associated register pressure off the hot warp-cooperative kernel.

When records are stored whole, the translation is one bulk pass. For each operation, the pass looks up the CRID in the cache index and replaces it with the resulting GRID. Once it finishes, every operation holds a GRID, and the executor reads cached records directly without further indexing.

With Epic's optional split-field layout, each field of a record has its own CRID and its own cache entry. EGAD allocates entries in admission order, so the k entries holding one record's k fields end up scattered across the cache. The executor needs all k GRIDs to perform a full-record operation, but cannot compute their positions from the record ID as Epic could with its dense layout. The translation therefore runs additional passes to gather these per-field GRIDs into a contiguous per-operation array.

3.7 Writeback

At the end of each epoch, each stager must propagate its partition's writes from the Execution Cache back to the Primary Store. The simplest design runs the writeback synchronously between epochs, blocking the next epoch from starting until every modified version slot has reached the Primary Store. The synchronous variant is straightforward but adds end-of-epoch latency proportional to the volume of modified version slots. EGAD therefore uses an asynchronous variant that reuses the same pipeline but overlaps it with the next epoch's work. We describe the synchronous design first, then show how the asynchronous variant builds on it.

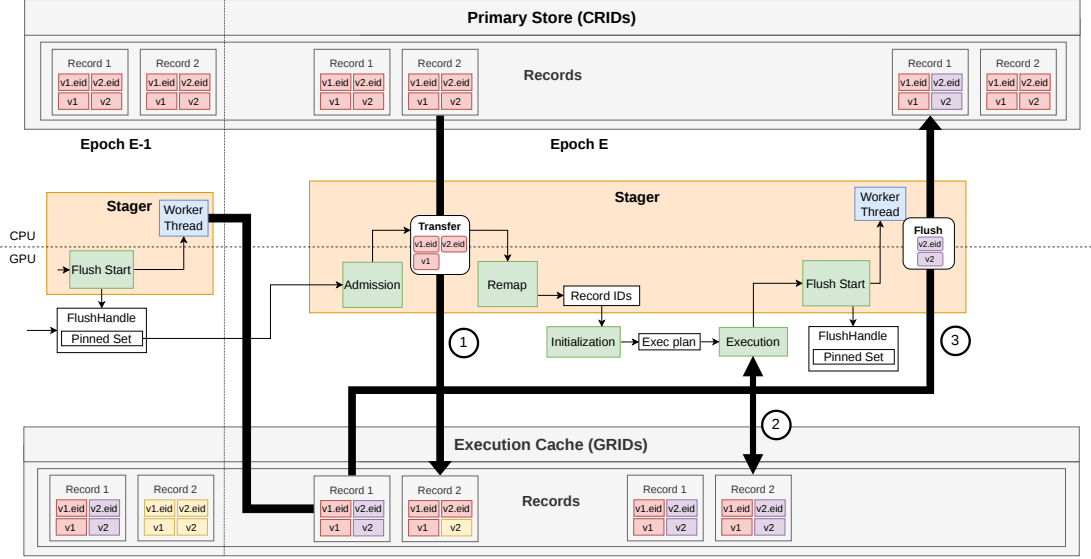


Figure 3: Asynchronous Writeback Across Two Epochs

3.7.1 Synchronous Writeback

The stager begins by collecting the GRIDs whose dirty flag is set for either version slot. For each such GRID, it gathers the modified version slot's epoch ID and value into a contiguous buffer. A single PCIe D2H transfer ships the buffer to the host, the CPU scatters each (epoch ID, value) pair into the corresponding version slot of the Primary Store, and the stager clears the dirty flags before the next epoch is allowed to begin. The writeback runs entirely on the main thread, so the next epoch only starts after it completes.

3.7.2 Asynchronous Writeback

Asynchronous writeback runs concurrently with the next epoch's pipeline, allowing higher throughput than synchronous writeback. While epoch E executes on the GPU, the writeback for epoch $E-1$ propagates $E-1$'s modifications to the Primary Store.

Three properties make this safe. First, each epoch's executor modifies exactly one of a record's two version slots (§2.5). The previous epoch's writeback ships the version slot modified in that prior epoch, so when consecutive epochs touch the same record, the current executor and the in-flight writeback land on different version slots and never conflict. Second, the eviction policy in the next epoch's admission skips any entry whose writeback has not yet finished (§3.4), so the unified admission/eviction operation cannot overwrite an entry whose data still needs to reach the Primary Store. Third, each writeback must finish before the next epoch ends, so the Primary Store stays at most two epochs behind the Execution Cache, and the bound also limits the work recovery must replay (§3.8).

Figure 3 traces the mechanism across epochs $E-1$ and E ,

with the Primary Store above the Execution Cache in each epoch column. Record 1 is modified in $E-1$ and is the subject of the writeback that runs during E . Record 2 is admitted by E and modified by E 's executor. At the start of $E-1$, both records hold their durable values in the Primary Store (red). Record 1's cache entry has version slot 2 marked as the $E-1$ modification (purple). Record 2's cache entry is empty (yellow).

At the end of $E-1$, the stager packages the data of Record 1's modified version slot and a pinned-set view of its cache entry, and a separate worker takes over the writeback concurrently with epoch E . The pinned set directs E 's admission to skip Record 1's cache entry. The worker scatters Record 1's version slot 2 to the Primary Store. Meanwhile, E 's main pipeline runs in parallel. Admission ① transfers Record 2 from the Primary Store into a previously-empty cache entry, shipping version slot 1's value and both epoch tags. MVCC initialization uses both tags to determine the read and write slots, and version slot 2's value is omitted because the executor will overwrite it. Remap rewrites operation CRIDs to GRIDs, and execution ② then writes Record 2's version slot 2, turning it purple.

By the end of E , the worker has finished Record 1's writeback ③, propagating its purple version slot 2 into the Primary Store. Record 2's Primary Store entry is still all red because its writeback has not yet started. Its cache entry holds the executor's purple write in version slot 2, which the stager will hand off for writeback during epoch $E+1$.

3.7.3 Avoiding Contention

During epoch E , the writeback of $E-1$ runs in parallel with E 's main pipeline, contending for four resources. The PCIe bus carries both the writeback's D2H transfer and admission's

H2D transfer. The H2D and D2H copy engines each serialize transfers in their direction, so per-stager writeback and admit transfers queue through a single engine in each direction. The GPU’s SMs are shared between the writeback’s gather kernel and the main pipeline’s kernels. Host DRAM bandwidth is used by both the writeback’s CPU-side scatter into the Primary Store and admission’s read from the Primary Store. The writeback workers run on the socket opposite the main pipeline, so their scatters reach the Primary Store over the inter-socket link rather than competing for the main pipeline’s cores, and the contention that remains is for the DRAM bandwidth of the store’s own node.

EGAD schedules the writeback to avoid concurrent access during admission, the only phase of E ’s pipeline that reads the Primary Store. Admission runs between indexing and MVCC initialization (Figure 1), so the writeback has E ’s non-admission phases as a non-contending window for Primary Store writes. Three cases arise based on the writeback’s volume relative to this window. When the writeback is small enough to finish before admission begins, no coordination is needed. When the writeback is so large it cannot fit even in E ’s full non-admission time, the worker overlaps with admission and accepts the contention. In the middle case, the writeback fits if it is split around admission.

EGAD handles the middle case by scheduling the worker to pause Primary Store writes when admission starts and resume when it finishes, so the host DRAM never serves a read and a write at the same time. EGAD implements the split where the middle case arises. Among the evaluated workloads that is YCSB. Its admit traffic is heavy enough that overlapping it would be costly, and its writeback is small enough to fit around it. The same pause also covers the small-writeback case, where its only cost is checking when admission starts and ends. TPC-C sits in the large case by construction, since its writeback spans most of the epoch while its admit volume stays small (§5.6), so its workers accept the overlap. Figure 13 in §5.6 shows this split schedule in a measured pair of epochs.

3.8 Recovery

Prior GPU-accelerated OLTP systems have no path from GPU memory to durable storage when the GPU loses state. EGAD’s architecture closes this gap by keeping records and the primary index recoverable from CPU-side state. EGAD implements this recovery scheme, and §5.7 measures its throughput cost. The failure model is a partial one. A GPU crash loses device state, and may take the host process down with it. The Primary Store and the Shadow Index are kept in durable host memory that outlives the process, so a freshly started process can re-map them and recover. A failure of the machine itself is instead covered by the standard durability techniques any in-memory database already requires, applied to that host-resident state.

The architecture admits recovery from a GPU crash by

replaying the two most recent epochs from CPU-resident state. While epoch E executes on the GPU, the previous epoch’s asynchronous writeback is still propagating its modifications to the Primary Store, and the epoch before that ($E - 2$) has writes already durable. A crash during E leaves the Primary Store consistent through end-of- $(E - 2)$. Recovery rolls back to that state and replays epochs $E - 1$ and E deterministically. Replay consumes those two epochs’ transaction inputs, so a deployment retains a rolling two-epoch input log in the same durable host memory. Our prototype instead regenerates the inputs from its deterministic workload generators, which serves the same role in the experiments.

Primary Store: Three properties of EGAD’s storage scheme make Primary Store rollback possible. First, Epic’s dual-version layout (§3.1) gives each record two version slots in the Primary Store, each slot carrying an epoch-ID tag alongside its value bytes. Second, the writeback writes a slot’s value bytes before its tag, so the tag changes only after the value is fully in place and acts as a commit marker for the slot. Finally, the asynchronous-writeback bound (§3.7.2) keeps the Primary Store at most two epochs behind the cache, so the values needed for rollback to end-of- $(E - 2)$ are already durable in the Primary Store.

Rollback must leave each record presenting the value it held at end-of- $(E - 2)$, the slot whose tag is the largest not exceeding $E - 2$. Recovery enforces this with one sweep over the store that demotes every slot whose tag belongs to a replayed epoch, leaving the surviving slot as each record’s newest. The unmodified executor then reads that slot as its base and directs replay’s writes at the demoted one. The two-epoch bound guarantees at most one slot per record needs demoting, and the commit marker guarantees the surviving slot’s value is fully written rather than partial. EGAD advances the durable epoch marker only after the previous epoch’s writeback has fully drained and before the next epoch’s writeback begins. The marker therefore always names a rollback target whose base values are still physically present in the Primary Store.

Shadow Index: A GPU crash also wipes the primary index, leaving the system unable to translate user keys into records. EGAD mirrors the primary index in CPU memory as a Shadow Index, so the index can be rebuilt and re-uploaded to the GPU after the crash. Because every key-to-CRID change happens in the indexing phase, the shadow only needs to track inserts and deletes, which it records in durable per-epoch logs (§3.9). CPU-side durability beyond a process restart follows standard in-memory-database recovery conventions [6, 8, 12, 13, 23].

3.9 Deletes

Deterministic batched execution makes deletes an indexing-phase operation. Every transaction’s operations are known

before the epoch runs, so the set of records an epoch deletes is part of the transaction input, and nothing needs to be discovered at execution time. Epic describes a delete design built on per-version validity flags, an end-of-epoch flag scan, and a record-to-key back-link array, but does not implement it [15]. EGAD implements deletes with none of that machinery. A delete contributes no operation to MVCC initialization or execution, so the execution kernels carry no delete path, and enabling deletes moves throughput by under one percent on both benchmarks (§5.7).

A delete takes effect at the epoch boundary, the granularity at which a batch commits. After the epoch’s indexing lookups complete, EGAD removes the deleted keys from the primary index in one bulk erase, so every transaction of the deleting epoch still resolves the key while no later epoch can reach it. The record’s CRID is never reused, and its Primary Store entry is never written again. The record is dead by unreachability.

In the Execution Cache, the stager flags a deleted record’s entry for reclamation, and the eviction scan of §3.4 drains flagged entries before it considers live residents. The flag is a preference rather than an override. A flagged entry still honors the eviction protection of the deleting epoch, whose earlier operations may read the record, and the writeback pin, since the record’s final modification must still reach the Primary Store. The same flag serves entries that are retired rather than deleted, records a workload retains but will rarely touch again, so every kind of logically dead entry leaves the cache ahead of records that may still be needed.

A delete never writes the Primary Store, so the record rollback of §3.8 is unaffected. A deleted record’s bytes stay in place and only the index forgets the key, which means recovery must reproduce the index change alone. The Shadow Index therefore gains a durable delete log, the dual of its insert log, and the recovery marker carries a delete cursor beside each insert cursor. Reconstruction replays the insert log and then the delete log up to the cursors durable at the rollback target. A record deleted at or before end-of- $(E - 2)$ is rebuilt absent. A record deleted during one of the two replayed epochs is rebuilt present, and replay re-executes its delete at the same point in the same epoch. Rolling back across a delete therefore revives the record without touching a single byte.

EGAD reclaims cache entries but defers physical reclamation of Primary Store rows. Record identifiers are never reused, so the store’s footprint grows with the total number of inserts rather than with the live set, a growth the evaluation’s bounded-length runs keep within the store’s provisioning. The natural extension recycles a deleted identifier two epochs after its delete, the same bound the asynchronous writeback and the recovery replay already obey. Epic derives the same horizon for pipelined index maintenance [15], and in-memory systems reclaim versions on similar epoch boundaries [10, 23]. We leave physical reclamation outside EGAD’s current scope.

3.10 Limitations

EGAD’s advantage depends on how many bytes the staging layer moves per useful operation, and the evaluation names the regimes where that ratio turns against it. At 1 KB records under low skew the staging cost exceeds the GPU’s execution advantage (§5.4.1), and full-record reads under field splitting amplify staging work tenfold (§5.4.3). Range scans are a further case of the same shape. The primary index is a hash table, so a scan decomposes into per-key lookups, and a scan-heavy workload such as YCSB-E would sweep per-epoch working sets far larger than those of the point-access workloads evaluated here, while presenting exactly the cross-epoch sequential pattern that FIFO eviction handles worst. The evaluated workloads contain no cross-epoch scans.

Three structural bounds follow from the design. The cache index is a flat array sized to the database rather than to the cache, so its device memory grows with database size, and databases an order of magnitude beyond those evaluated would need the sparse or hierarchical variant §3.1 notes. Partition capacities are fixed at startup, so a workload whose per-table working sets shift at runtime is served only as well as the initial split, pending the periodic resize §3.2 sketches. Deleted records return their cache entries but not their Primary Store rows (§3.9). Finally, EGAD targets a single GPU, and extending the stager across several devices is future work.

4 Implementation

This section describes how the key components of the staging architecture are implemented, beginning with how the per-table stager instances specialize to their tables.

4.1 Stager Instantiation

EGAD instantiates one stager per Execution Cache partition, so every table runs its own instance of the same stager implementation (§3.2). Three properties of a table decide how much of that machinery its instance exercises.

The first is whether the table fits its partition. A table granted enough capacity to hold itself entirely runs as a resident stager. It admits the whole table at load and never evicts, so its per-epoch pipeline reduces to bookkeeping and it pays no steady-state transfer cost. A table bounded below its size runs the full admit, evict, and writeback pipeline every epoch.

The second is whether the table grows through inserts. A growing table must size its Primary Store for the worst-case insert volume of the run, and committing physical memory for that reservation up front would waste tens of gigabytes. Growing stores therefore allocate pageable memory with huge pages requested through `madvise`, and the operating system backs only the pages the run touches. This is safe because steady-state transfers never issue DMA against the store itself. The admit gather and the writeback scatter stage

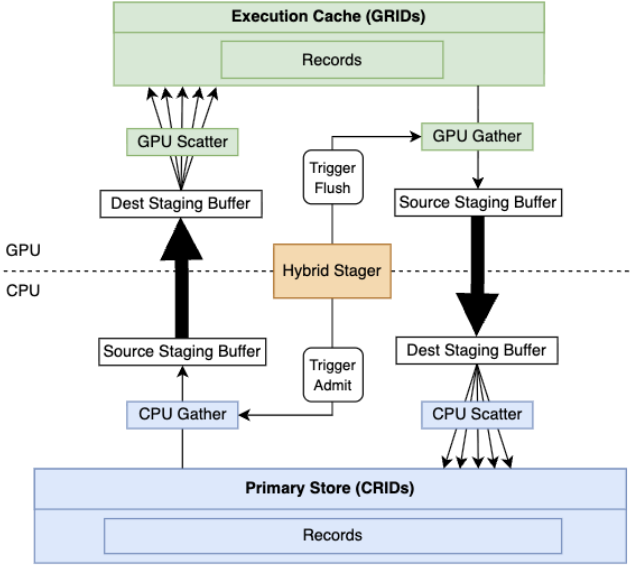


Figure 4: Scatter-Gather Subsystem

records through the pinned chunk buffers of §4.2, so the store is accessed only by CPU loads and stores. Fixed-size stores allocate pinned memory. When recovery is enabled, these allocations move into the durable Primary Store instead, accepting the up-front commitment in exchange for a store that survives the process (§3.8).

The third is whether the keyspace is dense and bounded. Such a table takes flat arrays for its primary and auxiliary indexes, replacing hash lookups with direct indexing.

In our benchmarks this instantiates as eight TPC-C stagers and one YCSB stager. TPC-C’s five fixed-size tables run resident, its growing NewOrder, Order, and OrderLine tables run the full pipeline, and OrderLine’s dense keyspace takes the flat primary index along with a flat customer-to-order auxiliary.

4.2 Scatter-Gather

Figure 4 shows the scatter-gather subsystem. Each call moves records between the CPU Primary Store and the GPU Execution Cache. An admit call gathers records from the Primary Store, ships them H2D, and scatters them into the cache. A writeback call gathers dirty slots on the GPU, ships them D2H, and scatters them into the Primary Store. A single call can move many megabytes in either direction.

Issuing a call as one PCIe transfer followed by one CPU pass would force a hard sequential dependency where one resource is idle while the other works. EGAD splits each call into chunks of bounded size and double-buffers them across two pinned host buffers that act as producer-consumer staging between the copy engine and the CPU loop. CUDA events synchronize the handoff so the CPU loop never reads a buffer mid-transfer. The chunk count is derived from each

call’s total byte volume against a fixed target chunk size, so a kilobyte-scale writeback runs as a single chunk with no chunking overhead while a multi-megabyte one gets enough chunks to keep the copy engine and the CPU loop both busy, with no per-table tuning. Thanks to this, the end-to-end latency approaches the maximum of PCIe time and CPU time rather than their sum, with the slower of the two bounding throughput.

The GPU gather and scatter kernels each have two variants. Records of at least 32 B use a warp-cooperative variant, where one warp processes one record and its 32 threads cover contiguous bytes within that record. That layout coalesces the warp’s accesses into one wide device-memory request per record, so the kernel runs at the memory bus’s sustained streaming bandwidth. Without coalescing, each lane would issue its own narrow request and the kernel would use only a fraction of available device-memory bandwidth. Records below 32 B use a per-thread variant since one record cannot fill 32 lanes anyway. The CPU scatter inside writeback runs on cores disjoint from the main thread’s gather, so DRAM scatter-writes do not compete with gather-reads for the same memory channels.

4.3 Per-Stager Parallelism

TPC-C’s eight stagers run `prepareEpoch` concurrently. The main thread enters an OpenMP `parallel` sections construct with one section per stager. Each section invokes its stager’s `prepareEpoch`, which issues its kernels on a per-stager CUDA stream so the GPU scheduler can co-schedule across the eight stagers rather than serializing them through the default stream. OpenMP nests one further level inside each section because the admit-side scatter-gather’s CPU gather opens its own parallel-for. The outer and inner team sizes are picked so their product stays close to the hardware thread count on the gather’s NUMA node, since exceeding that count oversubscribes the cores without adding parallelism. The inner size is kept small because the concurrent gathers are memory-bound on the same node, so a few threads per stager already reach the node’s DRAM bandwidth and wider inner teams only add contention. As YCSB only had a single table, it only has a single stager and runs `prepareEpoch` on the default stream with one OpenMP team for the gather.

YCSB runs each epoch’s writeback on a new `std::thread`, joining it before the next epoch begins. In TPC-C, each stager owns a persistent flush worker. The worker is a single `std::thread` that the stager creates at construction and keeps running for the lifetime of the run. The main thread submits each new writeback to the worker through a producer-consumer queue. This avoids the per-epoch thread-creation cost, which at TPC-C’s eight stagers per epoch would compete with the actual scatter work for CPU time. YCSB could use the same model, but with only one stager the cost is small enough that the simpler

spawn-and-join does not bottleneck throughput.

4.4 Synchronization

At several points in each epoch the GPU computes small values that the CPU needs in order to schedule the next phase. EGAD delivers these through a small region of mapped (zero-copy) host memory, introduced in §2.4. Because the kernel writes directly into this region, the main thread reads each value with a single CPU load. This matters during asynchronous writeback, since otherwise each read would queue behind the worker’s in-flight D2H on the copy engine and stall the main thread on a `cudaStreamSynchronize`.

CUDA’s default wait mode makes the worker busy-wait on the core that is about to run its scatter, consuming cycles and polluting the caches the scatter needs. EGAD overrides this with `cudaEventBlockingSync` on the worker’s events, so the OS deschedules the worker until the GPU signals. For the main thread, CUDA’s default lets the CPU enter a deep idle state between syncs and pay a wake-up cost on each one. EGAD overrides this with `cudaDeviceScheduleYield` on the device, so the main thread yields the CPU to other threads while waiting.

4.5 Asynchronous Writeback

At the end of each epoch, the stager launches a synchronous GPU gather kernel that writes each dirty slot’s selected version and value to a contiguous device buffer, then submits the rest of the writeback to the stager’s flush worker (§4.3). The worker drives the D2H copy and the CPU scatter through the writeback-side scatter-gather instance described in §4.2, advancing the writeback through its pipeline while the main thread starts the next epoch. At the end of the next epoch, before queueing its own writeback, the main thread issues a `sync_prev` join on the previous epoch’s worker, which enforces the two-epoch writeback bound from §3.7.2. When recovery is enabled, the stager still prepares each epoch’s flush set at the end of the epoch but hands it to the worker only after the recovery marker advances, which implements the marker discipline of §3.8 without adding work to the non-durable path. TPC-C runs this pattern in parallel across each growing-table stager.

EGAD runs the gather synchronously on all SMs even though the rest of the writeback is asynchronous. CUDA serializes kernel launches within a context, so submitting the gather to a separate stream does not overlap it with the next epoch’s kernels. Green contexts can partition the SMs to break that serialization, but the partial SM allocation slows the gather enough to wipe out the savings. Running on all SMs keeps the gather short enough to fit inside the per-epoch budget.

The worker’s CPU scatter and the main thread’s CPU gather both access the Primary Store on the same NUMA node, so

running them concurrently stalls both on the memory controller. EGAD splits the worker’s scatter in two around the gather, with the worker scattering the first half right after its D2H, waiting on a host flag the main thread sets when its gather completes, then scattering the second half. This is especially useful for YCSB, where the H2D admit traffic in some configurations is large enough to make the contention painful. TPC-C’s admit volume is small enough that the contention does not arise, so EGAD does not run the split in TPC-C.

5 Evaluation

We evaluate EGAD’s transaction throughput on the TPC-C and YCSB OLTP benchmarks against Epic’s two execution modes. *EPIC-GPU* runs indexing, MVCC initialization, and execution on the GPU, so the database must fit in device memory. *EPIC-CPU* moves execution to the CPU while leaving indexing and initialization on the GPU, supporting databases larger than device memory but paying a substantial throughput cost. We then analyze EGAD’s staging cost in detail and break down the gain from asynchronous writeback.

5.1 Experimental Setup

We run all experiments on a dual-socket Intel Xeon Silver 4410Y server with 24 physical cores (12 per socket, 48 logical threads with SMT) and 384 GB of memory, of which each socket has 64 GB of local DDR5-4000 and the rest sits on a CPU-less memory node our runs leave unused. The single accelerator we use is one NVIDIA RTX 5000 Ada (compute capability 8.9, 100 SMs, 32 GB of GDDR6 ECC, PCIe Gen4 x16, one copy engine per direction) attached to NUMA node 0. The operating system is Ubuntu 24.04 with Linux 6.8, and all binaries are built with gcc 13 and NVCC 12.9. We set the CPU frequency governor to `performance` and drop the page cache before every run.

TPC-C consists of five transactions, `NewOrder`, `Payment`, `OrderStatus`, `Delivery`, and `StockLevel`. As Epic does, we also run TPC-C NP, which consists of only `NewOrder` and `Payment`. Before each run, an `OrderLine` pre-warming pass admits the initial undelivered OL rows into the cache so steady state is reached within a few epochs. This speeds up our experiment runs, but does not change the steady-state performance. We run with $E = 50$ and report the median over $e \in [30, 50]$. TPC-C has no strict steady state at a fixed cache, since the growing `Order` and `OrderLine` tables keep deepening `Delivery`’s reach-back, so every TPC-C cell runs the same fixed transaction total and the window compares identical table-growth states across modes.

YCSB experiments use 20 M records of 120 B each, with each record holding ten 12 B fields. We use 120 B rather than the YCSB-spec 100 B so that field-split fields are 12 B, divisible by 4 for word-aligned GPU access.

Each epoch contains 100K transactions, and runs are 300 epochs long. Every experiment uses this epoch size. §5.4.4 measures the sensitivity and shows 100K is the measured optimum of both baselines while EGAD itself would keep gaining from larger epochs, so the fixed operating point favors the baselines.

By default, read operations return all ten fields of a record (a *full-record read*) and records are stored contiguously in one cache slot (no *field splitting*). §5.4.3 explores the alternatives, single-field reads and field splitting. Recovery is likewise disabled by default, and every experiment runs without it except §5.7, which enables it to measure its cost. EGAD runs use a 33.6% cache-to-database ratio, which holds cache pressure constant across configurations and isolates per-byte transfer cost from cache-hit-rate effects. The ratio is inherited rather than tuned. Sizing the cache to this GPU’s memory budget under the field-split slot geometry of §5.4.3 yields 6.72 M slots against the 20 M-record table, and every configuration adopts that same ratio. The one exception is the record-size comparison of §5.4.1, which instead matches both record sizes to its 1 KB configuration’s 52.7% ratio.

EGAD splits its CPU threads across the two NUMA nodes. The main thread and its OpenMP threads pin to node-0 cores via `numactl (OMP_NUM_THREADS=12)`, while the writeback worker pins itself to node-1 cores so it has its own CPU resources rather than sharing node-0 with the main pipeline. The worker still writes into the node-0 Primary Store, which the main thread reads during admit, so EGAD pauses the worker while admit is running (§3.7.3) to keep the two from sharing node-0 memory bandwidth at the same time. EPIC-CPU does not need this split. It uses one `std::thread` per physical core (`-c 24`). Adding more threads reduces throughput on this hardware. On YCSB-F at $\theta = 0.5$, `-c 32` measures 3.79 MTxn/s and `-c 48` measures 1.69, against 8.68 at `-c 24`. EPIC-CPU runs with no NUMA placement of its threads or memory, since NUMA-aware placement is part of EGAD’s design and not Epic’s. Its YCSB cells run an unmodified upstream Epic build, carrying only a GPU-architecture entry and a dependency header fix needed to compile on our hardware. The TPC-C cells run our fork’s binary, which we verified matches the upstream build’s throughput within 1% on the mixes both accept, and which adds a small-warehouse guard required to complete every swept run. The one configuration upstream’s CPU mode cannot run at all is field splitting, disclosed in §5.4.3 where it appears.

The cache starts empty, so early epochs admit records without evicting. Per-epoch admit volume settles into a steady rate once the hot records have accumulated and stay resident across epochs. We report the average throughput over the steady-state epoch window $e \in [250, 280]$, well past this warmup. Figure 5 plots the per-epoch admission volume behind this window choice. Admissions flatten within the first tens of epochs at every skew, and the slowest decay, at $\theta = 0.99$, settles by roughly epoch 100, so the window sam-

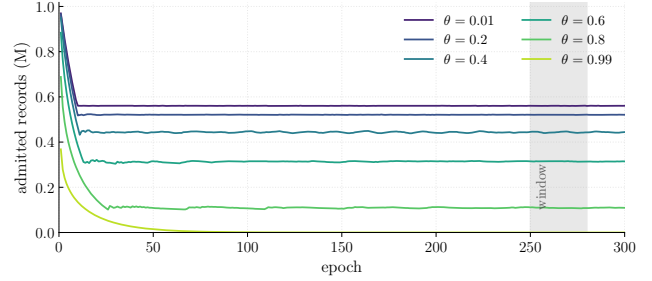


Figure 5: Per-Epoch Admission Volume, YCSB 20 M

ples the settled regime. Every measured cell in the evaluation aggregates at least three repetitions, and the whiskers in each figure span the repetitions.

5.2 TPC-C Throughput

Figure 6 sweeps TPC-C across warehouse counts $W \in \{4, \dots, 128\}$ and NP across $W \in \{4, \dots, 240\}$. The lower end excludes trivial warehouse counts ($W \leq 2$) where per-warehouse contention on the Warehouse, District, and Stock tables drops EPIC-GPU below EPIC-CPU [15] and would limit EGAD the same way since it uses the same GPU execution kernel. The upper ends differ by mix. EPIC-GPU must hold every table plus insert pools provisioned for the full run in device memory. On NP this footprint exhausts our 32 GB GPU above $W = 208$, so its line ends there while EGAD and EPIC-CPU continue to $W = 240$. On TPC-C, EPIC-GPU fits up to $W = 216$, and the sweep instead stops at $W = 128$ because the reach-back working set of Delivery and OrderStatus caps how far EGAD itself scales on this mix, discussed below.

EGAD beats EPIC-CPU at every swept W from $W = 8$ onward on both mixes. At $W = 128$, EGAD delivers 8.56 MTxn/s on TPC-C and 11.17 on NP, 1.41x and 1.36x over EPIC-CPU. On NP, EGAD sustains between 10.3 and 11.2 MTxn/s from $W = 128$ to $W = 240$, and past $W = 208$, where EPIC-GPU can no longer run, it stays above EPIC-CPU through the end of the sweep (1.32x at $W = 240$). EGAD pays the staging cost to keep execution on the GPU, and that GPU-execution advantage outweighs the staging cost at every swept warehouse count from $W = 8$ up.

Where EPIC-GPU fits, EGAD sits below it at a wider gap than YCSB showed at 5 M. At $W = 64$, EGAD reaches 48% of EPIC-GPU on TPC-C and 48% on NP, versus YCSB 5 M’s 52% to 97% across workloads. The wider gap on TPC-C reflects per-epoch admit traffic that YCSB 5 M does not pay. TPC-C’s NewOrder, Order, and OrderLine tables grow with execution. New rows go to fresh cache slots and do not require Primary Store transfers, but the stager still allocates the slot and updates the cache index for each one, scaling with per-epoch insert volume. YCSB at 5 M has no inserts in steady state and its cache holds the entire database after warmup, so its staging pipeline has nothing to do beyond bookkeeping.

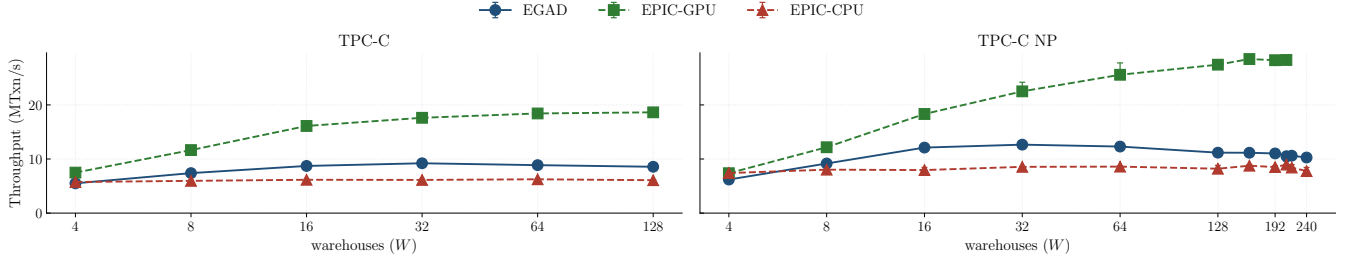


Figure 6: TPC-C and NP Throughput vs Warehouse Count

TPC-C’s Delivery, OrderStatus, and StockLevel transactions read existing OrderLine rows. In EGAD, those reads drive cache admission, and the writes from Delivery drive writeback traffic. NP removes all of this. Its NewOrder inserts go to fresh cache slots without pulling bytes from the Primary Store, and Payment does not access OL at all. The same OL traffic that EGAD pays as staging also appears in EPIC-GPU as memory loads in the execution kernel, showing up as a 39% NP advantage at $W = 64$ with no staging involved. Epic reports the analogous NP-vs-full-mix advantage under its native execution [15]. The same asymmetry sets how far each mix scales. NP’s inserts land in fresh cache slots and Payment touches no growing table, so EGAD’s device footprint is dominated by the fully resident tables, roughly 80 MB per warehouse, and the sweep continues past EPIC-GPU’s ceiling. The full mix instead needs the OrderLine and Order caches to hold its reach-back window. The resident tables crowd out those caches as W grows, and once the caches fall below the window the misses dominate the epoch and EGAD drops under EPIC-CPU by $W = 160$. This is the cache-capacity elbow of §5.4.2, reached by growing the database instead of shrinking the cache.

At $W = 4$, the EPIC-GPU lead over EPIC-CPU is narrow. EGAD shares Epic’s GPU executor and inherits the same per-warehouse contention, and the staging machinery itself adds overhead since the database fits in cache and staging saves no bandwidth. The two effects together put EGAD below EPIC-CPU at $W = 4$, at 0.96x on TPC-C and 0.84x on NP, its only swept point under the CPU baseline on either mix.

EPIC-CPU is otherwise roughly flat across W outside the small- W region (5.75 to 6.25 MTxn/s on TPC-C, 7.41 to 8.97 on NP, with both minima at $W = 4$). The CPU executor processes a fixed 100K transactions per epoch, and per-transaction work depends on the mix rather than on the warehouse count, so adding warehouses does not move EPIC-CPU’s throughput.

5.3 YCSB Throughput

We evaluate four YCSB workloads, A (50% read, 50% update), B (95% read, 5% update), C (100% read), and F (50% read, 50% read-modify-write). We evaluate all four at six Zipfian skews $\theta \in \{0.01, 0.2, 0.4, 0.6, 0.8, 0.99\}$ across two

regimes. The first uses 5 M records, sized so the database fits in VRAM, allowing comparison with EPIC-GPU. The second uses 20 M records at the canonical 33.6% cache-to-database ratio, which keeps EGAD under sustained cache pressure, allowing comparison with EPIC-CPU. EPIC-GPU holds this 20 M database as well, and §5.4.2 includes it as a ceiling reference. The configurations that genuinely exceed device memory are the 1 KB records of §5.4.1 and TPC-C NP at high warehouse counts (§5.2).

Figure 7 shows the result. At 20 M the comparison splits into two regimes. Below $\theta = 0.6$ the accessed records spread across the database, most operations miss the cache, and every epoch pays the full staging pipeline, while EPIC-CPU reads the same records straight from host memory. EPIC-CPU therefore holds most of the low-skew cells, with EGAD between 0.71x and 1.31x and nine of the sixteen cells at or below parity. From $\theta = 0.8$ the hot set concentrates into the cache, per-epoch admit traffic shrinks, and contended writes begin to serialize the CPU executor, so EGAD wins every cell, up to 3.10x on YCSB-B at $\theta = 0.99$ and 3.02x on read-only YCSB-C. The write-heavy YCSB-A and YCSB-F still drop in absolute terms at $\theta = 0.99$ because the hot records serialize the GPU executor, and EPIC-GPU at 5 M collapses the same way. EPIC-CPU falls further under that contention, which leaves EGAD 1.30x ahead on YCSB-F. On the read-heavy workloads the climb through high skew is steeper than EPIC-GPU’s because the staging cache concentrates onto the hot working set as skew rises, while EPIC-GPU already holds the entire database.

At 5 M, EGAD runs much faster than at 20 M because the entire database fits in the cache after warmup, eliminating per-epoch admit traffic (YCSB-A at $\theta = 0.6$ nearly doubles, a 92% gain from removing transfers). EGAD stays below EPIC-GPU because the staging pipeline still runs each epoch and writeback ships modified records back to the Primary Store. The gap to EPIC-GPU ranges from 9% on read-only YCSB-C to 48% on the write-heavy YCSB-A and YCSB-F at low skew.

5.4 Sensitivity

This section shows the impact of record size, cache size, read/write ratio, and epoch size by comparing EGAD’s per-

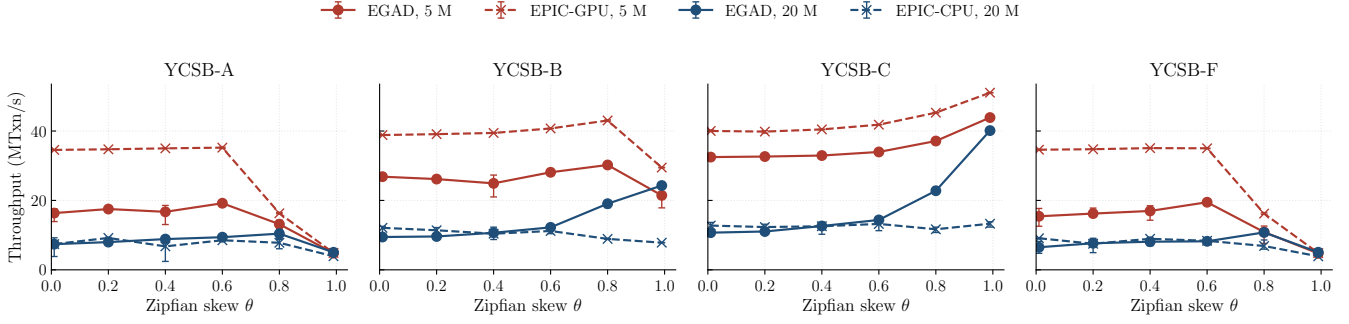


Figure 7: YCSB-A,B,C,F Throughput vs Different Skews

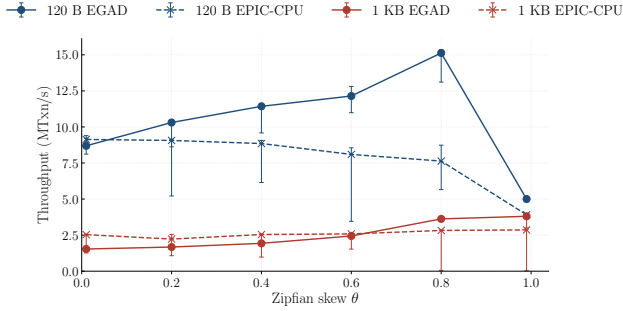


Figure 8: YCSB-F Throughput, 120 B vs 1 KB Records

formance with EPIC-CPU using the YCSB-F workload. Each study uses YCSB-F to isolate the effect from the cross-workload differences shown in §5.3. The epoch-size study widens the frame, adding TPC-C and the EPIC-GPU baseline, because it justifies the operating point the entire evaluation runs at.

5.4.1 Record Size

Record size sets how much data the staging pipeline ships per admit and per writeback. We test record-size sensitivity with YCSB-F at 120 B (used in §5.3) and 1 KB (the YCSB-spec default), each on a 20 M-database. At 1 KB the 20 M-record database exceeds device memory under Epic’s dual-version storage, so this is also a configuration EPIC-GPU cannot run at any cache setting. Both configurations use a matched 52.7% cache-to-database ratio so cache pressure stays equal across record sizes.

Figure 8 shows EGAD’s win over EPIC-CPU is far larger at 120 B than at 1 KB. At 120 B, EGAD beats EPIC-CPU at every skew from $\theta = 0.2$ up, peaking at 1.98x at $\theta = 0.8$, and sits just below parity at $\theta = 0.01$. At 1 KB, EGAD sits below EPIC-CPU through $\theta = 0.6$, a 39% deficit at the lowest skew, crosses above at $\theta = 0.8$, and reaches 1.33x at $\theta = 0.99$. The EPIC-CPU points are medians of ten runs each, since upstream’s CPU executor shows a recurring slow mode on our hardware that deepens with skew and record size, and a mean would sit between its modes.

The gap reflects how much data EGAD ships per record.

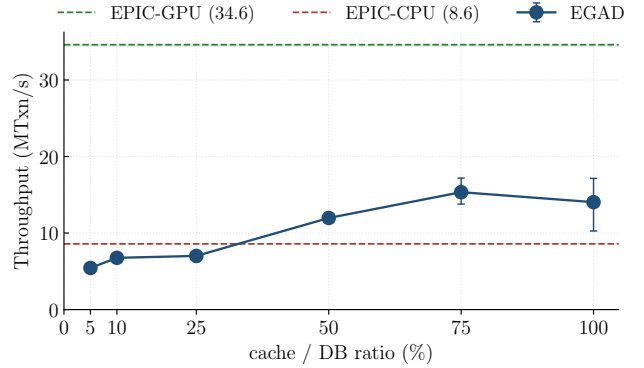


Figure 9: YCSB-F Throughput vs Cache/DB Ratio at $\theta = 0.5$

Each admit transfers a full record from the Primary Store to the cache, and each writeback ships the modified record back, so per-epoch transfer volume scales linearly with record size. At 120 B, transfer volume is small enough that the staging pipeline does not bottleneck the epoch. At 1 KB, per-epoch transfer volume is roughly 8x larger and the staging pipeline becomes the dominant epoch cost. EGAD only wins at 1 KB when the working set concentrates onto a small hot set at high skew as the per-epoch transfer data decreases.

At $\theta = 0.99$, EGAD on 120 B drops sharply to 5.00 MTxn/s (1.28x EPIC-CPU). This is the GPU-executor contention collapse from §5.3, where the hot record set serializes through the executor and execution becomes the dominant epoch cost.

5.4.2 Cache Size

Cache size controls how much of the working set the staging pipeline can keep resident across epochs. For YCSB-F at 120 B records and $\theta = 0.5$ on a 20 M-record database, we sweep the cache-to-database ratio across 5%, 10%, 25%, 50%, 75%, and 100%. EPIC-CPU and EPIC-GPU appear as horizontal reference lines at 8.59 and 34.6 MTxn/s respectively, since neither depends on EGAD’s cache size.

Figure 9 shows throughput climbing with cache size into a plateau region past 75%. This mirrors the read-heavy YCSB-B and YCSB-C curves in §5.3, where EGAD accelerated as higher skew concentrated the working set onto fewer records.

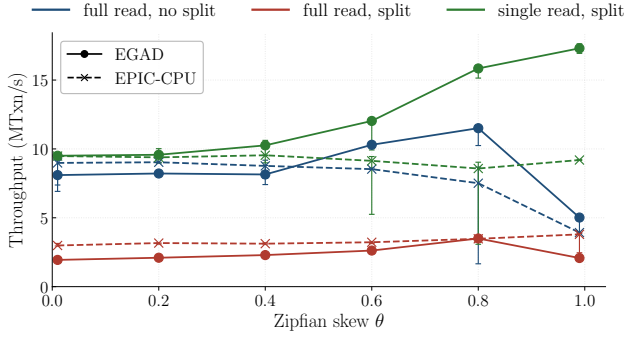


Figure 10: YCSB-F Throughput Across Three Storage Configurations

Here the working set stays fixed at $\theta = 0.5$ and the cache grows instead, with the same net effect on per-epoch admit volume, which drops once the cache holds the records the next epoch will touch. EGAD reaches 1.79x EPIC-CPU at 75% cache, and the 100% point measures within the plateau’s run-to-run band. The leveling past 75% means the $\theta = 0.5$ working set fits within three quarters of the database, so the remaining capacity buys little.

EGAD needs roughly half the database in cache to beat EPIC-CPU on this workload. At 25% and below the cache cannot hold the $\theta = 0.5$ working set and EGAD falls under the CPU baseline. At higher skew both thresholds shrink, since a smaller cache holds a more concentrated working set.

5.4.3 Read/Write Configuration

We test three combinations of read width and storage layout (defined in §5.1) against EGAD and EPIC-CPU at the canonical 33.6% cache-to-database ratio. The first uses full-record reads with no field splitting, the baseline. The second uses full-record reads with field splitting, where each read decomposes into ten field-level operations without reducing total transfer volume. The third uses single-field reads with field splitting, where each operation transfers only the targeted field rather than the whole record. Epic’s CPU mode does not support field-split storage, so the EPIC-CPU baselines for the two field-split configurations run our extension of its executor, which adds that path. The full-record configuration runs the unmodified upstream baseline.

Figure 10 shows the full-read-no-field-splitting configuration following the same contention regimes as §5.3, with EGAD between 0.90x and 0.93x of EPIC-CPU through $\theta = 0.4$, then 1.21x at $\theta = 0.6$ and 1.53x at $\theta = 0.8$ (11.51 vs 7.51 MTxn/s). The EPIC-CPU points that upstream’s slow mode reaches in this sweep again carry ten repetitions with median centers, matching the treatment in §5.4.1. The single-field configuration does better. Each read or write operation ships only the single field it targets rather than the whole record, dropping per-operation transfer volume to one-tenth of the full-read-no-field-splitting configuration. Since

transfer volume is EGAD’s bottleneck on this workload (per §5.4.1), this configuration stays at or above its baseline at every skew and pulls farthest ahead under contention, reaching 17.31 MTxn/s at $\theta = 0.99$ (1.88x EPIC-CPU’s 9.19).

With full-record reads and field splitting, EGAD loses to EPIC-CPU at every skew except $\theta = 0.8$. EGAD decomposes each full-record read into ten field-level operations, each requiring its own admit, MVCC entry, and cache slot. Epic notes the same penalty for full-record reads under field splitting, observing a 10x increase in initialization-phase read operations and a corresponding drop in read performance [15]. EPIC-CPU does not enter the staging pipeline at all, so at low skew this amplified staging cost overshadows the GPU’s execution-time advantage and EGAD drops below EPIC-CPU. Like the 1 KB low-skew result in §5.4.1, this is a regime where the staging pipeline must do more work per useful read than the workload’s effective data needs justify.

5.4.4 Epoch Size

Epoch size sets the batch every stage of the pipeline amortizes over. Figure 11 sweeps it from 5K to 400K transactions on YCSB-F at $\theta = 0.5$ and on TPC-C at $W = 64$, against both baselines. Total transactions per run are held fixed, so every measurement window sits at the same table-growth state regardless of epoch size, and cache capacities are pinned at the canonical configuration so capacity does not covary with the sweep. The right panel replots the same YCSB-F measurements against the average latency each epoch size implies. The latency is derived rather than measured, as 1.5x the epoch wallclock, Epic’s convention [15]. A transaction arrives at a random moment while its batch is collected, waits half an epoch for the batch to close, and then one epoch for the batch to execute. Under asynchronous writeback its modifications reach the Primary Store up to one epoch later still, so durability trails results by one epoch and averages 2.5x the epoch wallclock. The same epoch size takes each system a different time, so the curves do not share x positions and faster systems sit further left. Each curve traces its system from 5K epochs at its left end to 400K at its right end, with a ring marking the 100K point, and is read against a latency budget. For any budget on the x axis, the best throughput a system offers within that budget is the highest point of its curve at or to the left of it.

Both baselines measure best at 100K, consistent with Epic’s report [15]. EPIC-GPU peaks at exactly 100K on both workloads and declines beyond it, and EPIC-CPU stays nearly flat, gaining under 6% from 100K to 400K. EGAD instead keeps rising through the whole swept range, gaining 28% from 100K to 400K on YCSB-F and 9% on TPC-C. Larger epochs amortize EGAD’s fixed per-epoch staging costs, and they also shrink per-transaction admission volume, since repeated accesses to a record inside one epoch admit it once. At $\theta = 0.5$, the low-skew regime where §5.3 shows near-parity,

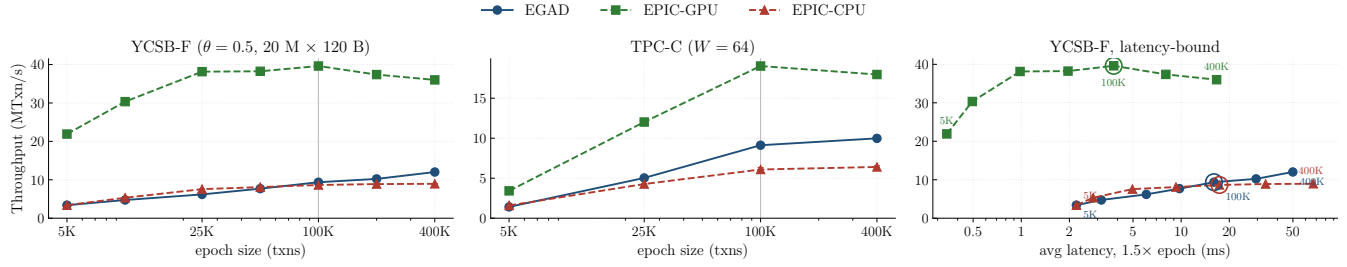


Figure 11: Throughput vs Epoch Size, and YCSB-F Throughput vs Latency

Table 1: Device memory by structure (GB), with driver-reported totals.

Structure (what it scales with)	YCSB 20 M	TPC-C 128 W
Cache entries (capacity, 256 B)	1.60	17.58
Staging metadata (capacity, ~60 B)	0.36	4.19
Cache index + insert map (rows, 5 B)	0.09	0.58
Primary index (keys)	0.19	1.85
Version scratchpad (epoch operations)	0.12	0.10
Auxiliary indexes (TPC-C keyspaces)	—	0.35
Transaction + planner buffers (epoch size)	0.98	1.34
Per-epoch scratch + CUDA runtime	0.67	2.66
EGAD total (measured)	4.01	28.64
EPIC-GPU total (measured)	8.68	21.99

EGAD’s lead over EPIC-CPU therefore grows from 1.08x at 100K to 1.34x at 400K. In the latency view, EPIC-GPU offers its maximum near 4 ms and loses throughput beyond it, EPIC-CPU saturates by roughly 17 ms, and EGAD alone keeps converting additional latency budget into throughput through 50 ms.

These curves are why the evaluation fixes 100K (§5.1). It is the measured optimum of both baselines and the operating point Epic reports at [15], and it understates EGAD, which reaches its swept maximum only at four times that epoch size. The small end costs EGAD most of its throughput, 3.5x to 7.1x below its 400K rate at 5K epochs, because the fixed per-epoch costs stop amortizing, so deployments with latency targets in the low milliseconds sit in the baselines’ regime rather than EGAD’s. On TPC-C the sweep is also bounded by device memory, since Epic provisions the growing tables’ projections up front in proportion to run length times epoch size, which caps how far a latency-insensitive deployment can push the epoch size before that provisioning no longer fits.

5.5 Memory Footprint

Table 1 breaks down EGAD’s device memory for the two principal configurations. Every row derives from the allocation sites, and the totals are read back from the driver at run time, so the rows reconcile against what the GPU actually holds. The structures sized by cache capacity dominate, and the remainder row collects per-epoch scratch buffers together with the CUDA runtime’s own footprint.

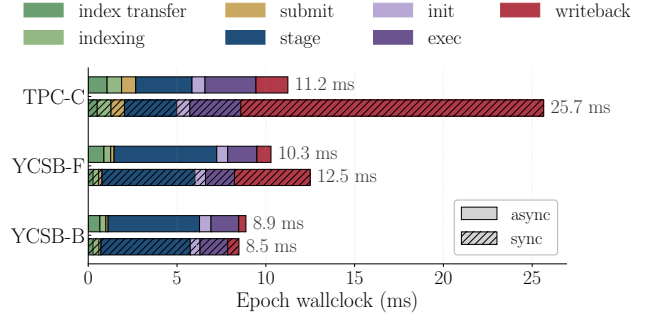


Figure 12: Per-Phase Epoch Breakdown, Async vs Sync

The rows that scale with the database rather than the cache cost EGAD 15 B per row, 4 B of cache index, 1 B of insert map, and 10 B of hash slot at the index load factor. EPIC-GPU instead stores the row itself, 256 B of dual-version record plus 128 B of scratchpad version plus the same hash slot, roughly 394 B per row. At the same 20 M-record YCSB database EGAD’s device footprint is 4.0 GB against EPIC-GPU’s 8.7 GB, and growing the database moves EGAD’s device cost 26x more slowly. The database-proportional metadata alone would not exhaust this GPU until roughly 2.1 billion rows, about 100x the evaluated scale, which quantifies the headroom behind the flat cache index and marks where the sparse variant §3.1 notes becomes necessary.

TPC-C inverts the totals, 28.6 GB for EGAD against 22.0 GB for EPIC-GPU at $W = 128$, because EGAD holds the fixed tables resident, grants the growing tables bounded caches, and keeps per-entry staging metadata for all of it. The difference is in what grows. Both systems size their insert-receiving tables to the run’s projected inserts, and each projected row costs EPIC-GPU 384 B of record and scratchpad where it costs EGAD 5 B of index metadata, so higher warehouse counts and longer runs reach EPIC-GPU’s ceiling first. That is the boundary the NP sweep crosses at $W = 208$ (§5.2).

Field splitting is the layout’s worst case for slot efficiency. Each 12 B field occupies its own pair of 128 B slots, 24 B of useful data per 256 B entry, a cost §5.4.3 accepts because it cuts transferred bytes tenfold on single-field operations, and transfer volume rather than device capacity is the binding resource in that regime.

5.6 Asynchronous Writeback

Asynchronous writeback overlaps each epoch’s writeback with the next epoch’s main-thread pipeline. The speedup from overlap alone is bounded by the ratio of writeback time to the rest of the epoch, with 2x as the optimum when the two are equal. The async path can land past that bound because its writeback is also faster than sync mode’s. It sorts each flush set so the worker scatters into the Primary Store in ascending record order, where sync mode walks the set in cache order, so the async worker finishes the same writeback in less time. Contention with the next epoch’s admit traffic pushes in the other direction, since both the writeback worker’s CPU scatter and the admit’s CPU gather access the Primary Store. Figure 12 shows per-phase breakdowns for three cells spanning the writeback-share range. TPC-C at $W = 128$ is the writeback-heaviest case, YCSB-F at $\theta = 0.5$ is moderate, and YCSB-B at $\theta = 0.5$ is writeback-lightest. We omit YCSB-A and YCSB-C because their async/sync ratios match YCSB-F and YCSB-B respectively, and three cells suffice to span the trade-off range.

TPC-C shows the largest gain. Sync mode runs 25.65 ms per epoch and async cuts this to 11.24 ms, a 2.28x speedup. Its sync writeback (17.06 ms) exceeds the rest of the sync epoch (8.59 ms), so overlap hides the non-writeback work inside the writeback window, and the async path’s faster ordered scatter supplies the remainder of the gain. The worker runs writeback throughout the next epoch and overlaps with admit, but TPC-C’s per-epoch admit volume is small because the growing tables’ inserts use fresh cache slots (per §5.2). The result is low contention even when the writeback bar spans the entire epoch.

YCSB-F shows a smaller gain at 1.21x (sync 12.49 ms, async 10.29 ms). Its sync writeback (4.26 ms) is smaller than the rest of the sync epoch (8.23 ms), so only the writeback time can be hidden. YCSB-F has a much larger per-epoch admit volume than TPC-C, but the worker pauses its scatter during admit and resumes after, effectively splitting the writeback into two chunks around the admit (§3.7.3). This keeps the contention overhead small enough to leave a meaningful win.

Figure 13 makes this schedule visible for the same YCSB-F cell, plotting two consecutive steady-state epochs from the per-phase timestamps in the run’s log. In the asynchronous panel, the worker lane carries each epoch’s writeback while the next epoch occupies the main thread. The writeback runs as its two halves around a pause, and the pause ends together with the admission transfer whose Primary Store gather it yields to, marked by the dotted line. The synchronous panel runs the same pipeline with the writeback inline after execution, and the epoch stretches from 10.29 to 12.49 ms.

YCSB-B shows a small net loss (sync 8.49 ms, async 8.88 ms, 0.96x). Its sync writeback is only 0.64 ms (7% of the epoch), so there is little for the worker to hide, and the

contention tax outweighs it. The worker’s CUDA API calls and node-1 scatter inflate the other phases, with index transfer growing from 265 to 656 us in particular. Asynchronous writeback stays the default because the loss is small where writeback is negligible and the gain is large where writeback dominates.

5.7 Recovery Cost

§3.8 keeps the Primary Store recoverable by backing it with durable memory and mirroring the primary index into the Shadow Index, both of which survive the process restart following a GPU crash. We measure the throughput cost of maintaining that copy by running TPC-C with recovery enabled and comparing against the non-recovery configuration on the same build across the warehouse sweep. Figure 14 shows the result. Recovery costs between 12% and 19% of throughput across $W \geq 8$, and EGAD with recovery stays above EPIC-CPU across that whole range, 1.15x at $W = 128$, so most of the capacity advantage from §5.2 survives with recovery enabled. At $W = 4$ the staging machinery already gives up EGAD’s margin and recovery lands below the CPU baseline. A deployment that would otherwise run EPIC-CPU gets better throughput from EGAD with recovery at $W = 8$ and above, and gains recovery from GPU crashes, which Epic offers in neither mode.

The cost comes mostly from maintaining the durable Shadow Index during the indexing phase, where each epoch appends its new key-to-CRID assignments to the durable copy. The writeback path needs no extra work, since the dual-version commit marker that makes Primary Store rollback possible (§3.8) already leaves each modified record durable. The marker discipline does constrain scheduling, since the marker may advance only after one epoch’s writeback drains and before the next epoch’s begins, and TPC-C pays a few percent of throughput for holding its workers’ submission to that point. The overhead is also flat in run length, because recovery always replays exactly two epochs rather than growing with a checkpoint interval. Because this cost tracks insert volume, workloads without steady-state inserts avoid almost all of it. The YCSB workloads read and update a fixed key set with no inserts, so enabling recovery moves their throughput by under 0.5% even at the low skew where writeback volume is largest, leaving TPC-C with its growing tables as the stress case. Recovery time itself is small. On YCSB-F with the 20 M record Primary Store, a process relaunched after an injected GPU fault is caught up with the pre-crash state about five seconds after launch, and most of that time goes to re-mapping the 41 GB durable store as pinned memory. The version rollback demotes up to 480 K Primary Store slots in 54 ms, and re-executing the two epochs inside the writeback bound adds a few hundred milliseconds, against 13 seconds for a cold start that must repopulate the database before its first epoch.

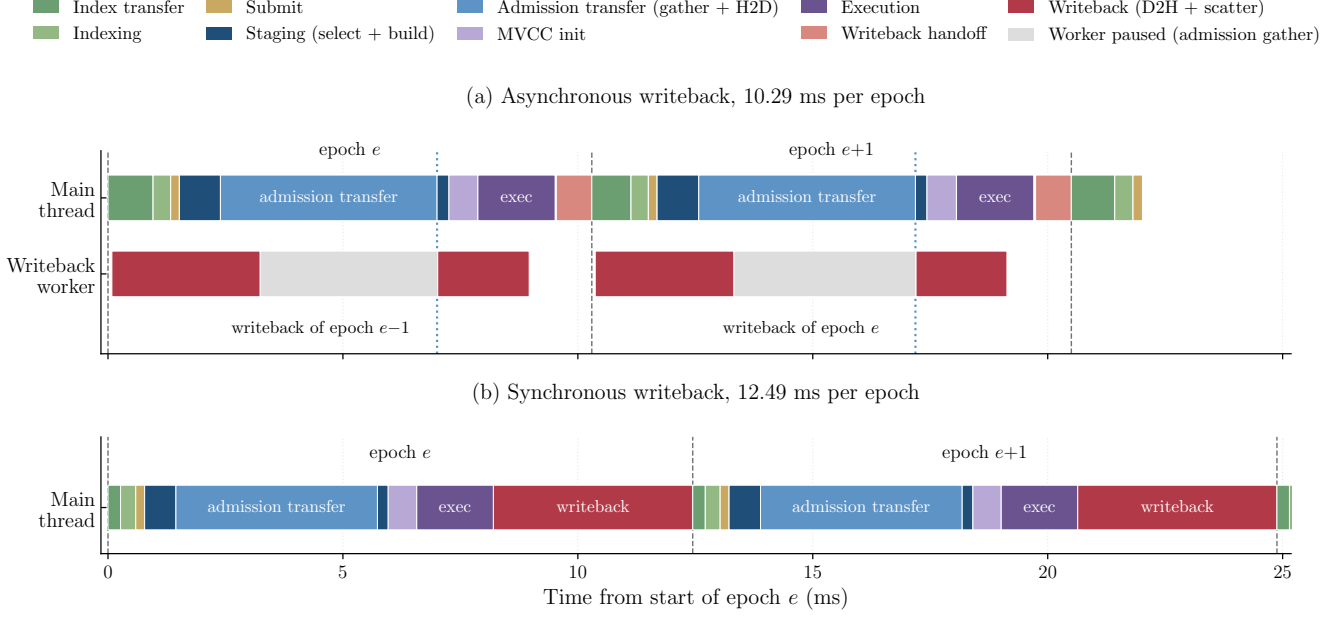


Figure 13: Measured Epoch Timeline, Async vs Sync Writeback (YCSB-F)

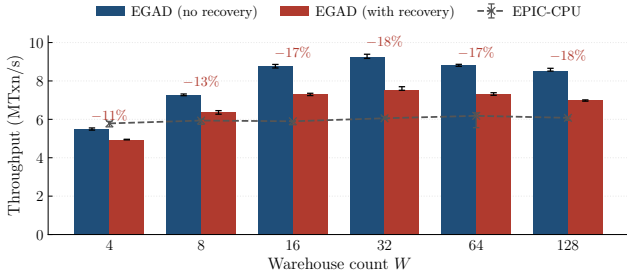


Figure 14: TPC-C Throughput With and Without GPU-Crash Recovery

Deletes (§3.9) add one more entry to this durable path, an append of the epoch’s deleted keys to the delete log during indexing. TPC-C’s deletes come from Delivery, which removes the NewOrder row it consumes, about 43K removals and log appends per epoch at $W = 128$. Their cost is not visible in our measurements. Enabling deletes moves TPC-C throughput by under one percent in every configuration we measured, and the YCSB workloads of §5.3, which contain no deletes, measure within one percent of a build without the delete path. Beyond cost, we stress the delete path with a sliding-window workload that inserts records at the head of the key space and deletes the same number from the tail each epoch, holding the live set constant while every key eventually enters and leaves the index, and we validate crash recovery under it.

6 Conclusions

GPU-accelerated OLTP delivers high throughput but has been bounded by device memory, and it has offered no path to recovery when the GPU loses its state. We presented EGAD, which addresses both limits with a single architectural change. The full database lives in CPU memory as the Primary Store, and each epoch’s working set is staged into a GPU-resident Execution Cache. Deterministic batched execution makes that working set exactly computable before execution begins, so only the records an epoch touches must fit on the GPU, and database capacity is bounded by host memory rather than VRAM.

Epic’s dual-version storage serves two roles in this design. Shipping one version per record halves transfer traffic in each direction, and because consecutive epochs write disjoint version slots, each epoch’s writeback runs concurrently with the next epoch’s execution. The same slot discipline keeps the Primary Store at most two epochs behind the GPU, which caps the work recovery must redo. A GPU crash, even one that takes the process down with it, costs a rollback and two epochs of replay from CPU-resident state, and deletes are handled at the same epoch boundaries.

On TPC-C, EGAD reaches 8.56 MTxn/s (1.41x EPIC-CPU) at $W = 128$ on the full mix, where asynchronous writeback contributes a 2.28x speedup over a synchronous baseline, and on NP it sustains 10.3 MTxn/s through $W = 240$, past the point where Epic’s GPU mode exhausts device memory. On YCSB the advantage over EPIC-CPU grows with contention, reaching 3.10x on YCSB-B, while EPIC-CPU stays competitive at low skew. Where EPIC-GPU fits in device memory,

EGAD sits below it, paying the staging cost in exchange for capacity beyond VRAM and recovery from GPU crashes. Enabling recovery costs 12% to 19% of TPC-C throughput, and EGAD with recovery stays above EPIC-CPU from $W = 8$ up, while a relaunched process catches up with the pre-crash state in seconds. Two directions remain. Recycling deleted identifiers on the two-epoch bound the writeback already obeys would let the Primary Store’s footprint track the live set, and extending the stager across multiple GPUs is a natural next step once a single device’s transfer path saturates.

References

- [1] Nils Boesch and Carsten Binnig. GaccO - A GPU-accelerated OLTP DBMS. In *Proceedings of the 2022 International Conference on Management of Data, SIGMOD ’22*, pages 1003–1016, New York, NY, USA, June 2022. Association for Computing Machinery.
- [2] Cristian Diaconu, Craig Freedman, Erik Ismert, Per-Ake Larson, Pravin Mittal, Ryan Stonecipher, Nitin Verma, and Mike Zwilling. Hekaton: SQL Server’s Memory-Optimized OLTP Engine. In *Proceedings of the International Conference on Management of Data - SIGMOD*, pages 1243–1254. ACM, 2013.
- [3] Jose M. Faleiro and Daniel J. Abadi. Rethinking serializable multiversion concurrency control. *Proceedings of the VLDB Endowment*, 8(11):1190–1201, July 2015.
- [4] Jose M. Faleiro, Daniel J. Abadi, and Joseph M. Hellerstein. High performance transactions via early write visibility. *Proceedings of the VLDB Endowment*, 10(5):613–624, January 2017.
- [5] Bingsheng He and Jeffrey Xu Yu. High-throughput transaction executions on graphics processors. *Proceedings of the VLDB Endowment*, 4(5):314–325, February 2011.
- [6] JEDEC Solid State Technology Association. JESD248: DDR4 NVDIMM-N Design Standard. <https://www.jedec.org/standards-documents/docs/jesd248>, 2023.
- [7] Kangnyeon Kim, Tianzheng Wang, Ryan Johnson, and Ippokratis Pandis. ERMIA: Fast Memory-Optimized Database System for Heterogeneous Workloads. In *Proceedings of the 2016 International Conference on Management of Data, SIGMOD ’16*, pages 1675–1687, New York, NY, USA, June 2016. Association for Computing Machinery.
- [8] Hideaki Kimura. FOEDUS: OLTP engine for a thousand cores and NVRAM. In *SIGMOD*, pages 691–706, 2015.
- [9] Yinan Li, Bailu Ding, Ziyun Wei, Lukas M. Maas, Momin Al-Ghosien, Spyros Blanas, Nicolas Bruno, Carlo Curino, Matteo Interlandi, Craig Peeper, Kaushik Rajan, Surajit Chaudhuri, and Johannes Gehrke. Scaling GPU-accelerated databases beyond GPU memory size. *Proceedings of the VLDB Endowment*, 18(11):4518–4531, 2025.
- [10] Hyeontaek Lim, Michael Kaminsky, and David G. Andersen. Cicada: Dependably Fast Multi-Core In-Memory Transactions. In *Proceedings of the 2017 ACM International Conference on Management of Data, SIGMOD ’17*, pages 21–35, New York, NY, USA, May 2017. Association for Computing Machinery.
- [11] Yi Lu, Xiangyao Yu, Lei Cao, and Samuel Madden. Aria: A fast and practical deterministic OLTP database. *Proceedings of the VLDB Endowment*, 13(11):2047–2060, July 2020.
- [12] Nirmesh Malviya, Ariel Weisberg, Samuel Madden, and Michael Stonebraker. Rethinking main memory OLTP recovery. In *ICDE*, pages 604–615, 2014.
- [13] C. Mohan, Don Haderle, Bruce Lindsay, Hamid Pirahesh, and Peter Schwarz. ARIES: A transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging. *ACM Transactions on Database Systems*, 17(1):94–162, 1992.
- [14] Thomas Neumann, Tobias Mühlbauer, and Alfons Kemper. Fast Serializable Multi-Version Concurrency Control for Main-Memory Database Systems. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data, SIGMOD ’15*, pages 677–689, New York, NY, USA, May 2015. Association for Computing Machinery.
- [15] Shujian Qian and Ashvin Goel. Massively parallel multi-versioned transaction processing. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 765–781, Santa Clara, CA, July 2024. USENIX Association.
- [16] Dai Qin, Angela Demke Brown, and Ashvin Goel. Caracal: Contention Management with Deterministic Concurrency Control. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles, SOSR ’21*, pages 180–194, New York, NY, USA, October 2021. Association for Computing Machinery.
- [17] Alexander Thomson, Thaddeus Diamond, Shu-Chun Weng, Kun Ren, Philip Shao, and Daniel J. Abadi. Calvin: Fast distributed transactions for partitioned database systems. In *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data, SIGMOD ’12*, pages 1–12, New York, NY, USA, May 2012. Association for Computing Machinery.

- [18] Stephen Tu, Wenting Zheng, Eddie Kohler, Barbara Liskov, and Samuel Madden. Speedy transactions in multicore in-memory databases. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles, SOSP '13*, pages 18–32, New York, NY, USA, November 2013. Association for Computing Machinery.
- [19] Yingjun Wu, Joy Arulraj, Jiexi Lin, Ran Xian, and Andrew Pavlo. An empirical evaluation of in-memory multi-version concurrency control. *Proc. VLDB Endow.*, 10(7):781–792, mar 2017.
- [20] Juncheng Yang, Yazhuo Zhang, Ziyue Qiu, Yao Yue, and K. V. Rashmi. FIFO queues are all you need for cache eviction. In *SOSP*, pages 130–149, 2023.
- [21] Bobbi W. Yogatama, Weiwei Gong, and Xiangyao Yu. Scaling your hybrid CPU-GPU DBMS to multiple GPUs. *Proceedings of the VLDB Endowment*, 17(13):4709–4722, 2024.
- [22] Yichao Yuan, Advait Iyer, Lin Ma, and Nishil Talati. Vortex: Overcoming memory capacity limitations in GPU-accelerated large-scale data analytics. *Proceedings of the VLDB Endowment*, 18(4):1250–1263, 2024.
- [23] Wenting Zheng, Stephen Tu, Eddie Kohler, and Barbara Liskov. Fast databases with fast durability and recovery through multicore parallelism. In *OSDI*, pages 465–477, 2014.